LinkedIn AI Agent: structure and serverless operation

Oct 29, 2026 · @Guhaprasaanth N

Summary

The agent can run with no server: two short GitHub Actions workflows, one that composes the post and one that checks your Telegram taps every 30 minutes, with Telegram's message queue holding your replies in between. It works end to end today and published 3 posts on Oct 29, but its quality gates leak: the latest post went live with a placeholder source, a failed fact-check and an infographic with overlapping text.

It is a single-process Python pipeline (about 2,300 lines across 6 modules, no agent framework). An LLM researches one allowed source with web search, writes the post, draws an SVG infographic, and grades both. A human approves on Telegram, then it posts to the LinkedIn feed through the official API and to 8 LinkedIn groups through browser automation.

The three things to fix first:

1. **Claude is not running.** The Anthropic account is out of credits, so every call since mid-afternoon has silently fallen back to gpt-4o, a weaker writer, designer and reviewer.
2. **Custom-topic mode does no real research.** The prompt hands the model a fake MDN source dated 2024-01-01, and the fact-check result is ignored for this mode. This is the mode the project goal ("research a private topic") depends on.
3. **Group posting is high risk.** Posting the same text to 8 groups via Playwright with a stealth flag breaks LinkedIn's automation rules and can get the account restricted.

Tasks: what one run does

One run makes at most one post and takes up to 3 attempts; each failed gate feeds its notes back into the next attempt.

1. **Pick a source.** Choose one of the 78 allowed sources at random, skipping any used in the last 3 days. With a custom topic, the topic is the subject and a source is still picked to scope the search.

2. **Research and write.** One model call with web search limited to that source finds one item from the last 7 days and returns JSON: the post, a headline, the sources with dates and the claim each supports, and an infographic idea. If nothing is fresh, the next source is tried.
3. **Code checks.** 700 to 3,000 characters, 3 to 7 emojis and none adjacent, no repeated words or stray JSON, every source on the allow-list, dated within 7 days and never posted before.
4. **Design the infographic.** A designer call returns an SVG (1080x1350 portrait). Code checks it for safety and legibility (no scripts or external links, fonts of 22 or more, at least 3 labels), then CairoSVG renders the PNG. The designer gets 2 tries.
5. **Review.** Three independent editorial reviews look at the post and the rendered image and score six criteria; every score must be 4 or more. A separate fact-check re-searches the cited sources. All must pass.
6. **Human approval on Telegram.** The image and post go to your chat with Approve and Reject buttons, a reminder every 20 minutes and a 3 hour expiry. Reject offers revise with notes, suggest topics, or skip.
7. **Publish.** Post to your LinkedIn feed through the API, try to add the sources as a first comment, then post to the 8 configured groups through the browser bot.
8. **Log.** Append the post to `posted.json` so its URLs are never reused.

Components

Everything is plain Python calling the Anthropic and OpenAI SDKs directly; there is no agent framework, no tool loop of your own, and no tests.

File	Lines	What it does
`agent.py`	1,086	The whole pipeline: source picking, writer, designer, reviewers, fact-check, code checks, Telegram flows, LinkedIn publishing, CLI
`telegram_review.py`	644	Approve/Reject buttons, reminders, 3 h timeout, revise and topic-switch flows, the 24/7 command listener, draft state in `state/drafts.json`
`linkedin_groups.py`	234	Playwright bot that opens each group page and clicks through the composer, using a saved `browser_profile/` login

File	Lines	What it does
`infographic.py`	154	SVG safety and legibility checks, CairoSVG render to 1080x1350 PNG, an unused animated-GIF renderer
`linkedin.py`	93	LinkedIn REST client: image upload, post, comment
`linkedin_auth.py`	68	One-time OAuth login that writes the token into `.env`
`config.yaml`	132	Voice, 700 to 3,000 character range, 3 to 7 emojis, review thresholds, 79 allowed sources, 8 target groups

Models and tools. Primary model is `claude-sonnet-5-5` with Anthropic's server-side `web_search_20250305` tool, restricted to one source domain per run. Fallback is `gpt-4o` via OpenAI's Responses API with `web_search_preview`, which cannot be restricted to domains, so the allow-list becomes a prompt instruction only.

Prompts. Five roles, all inline in `agent.py`: writer (two variants, source mode and topic mode), designer (a long style guide for SVG output), editorial reviewer (six criteria scored 1 to 5, sees the rendered PNG), fact-checker (re-searches the cited sources), and a topic scout for suggestions.

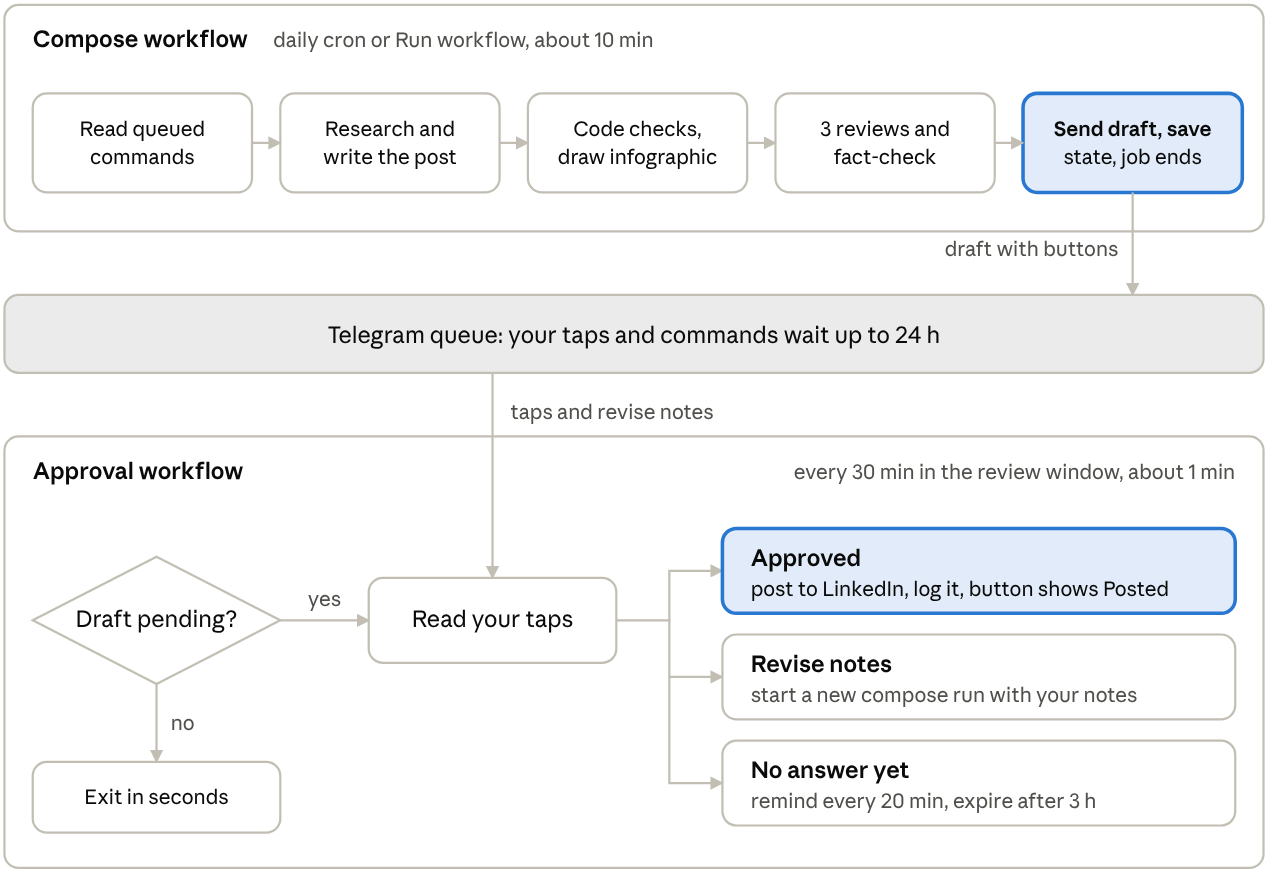
Running it. Three entry points: `agent.py` once a day (launchd plist or GitHub Actions cron), `agent.py --listen` as a 24/7 Telegram bot daemon, and `--direct` for your own text.

Dependencies. anthropic, openai, python-dotenv, pyyaml, requests, cairosvg, playwright. No versions are pinned.

Running on GitHub Actions with no server

The agent can run entirely as two short GitHub Actions workflows, with Telegram's message queue standing in for an always-on bot. The current workflow already runs on a daily cron, but it keeps one job open for up to 3 hours waiting for your tap. It also has no Telegram progress messages and no way to take your commands.

Two short jobs, no server: Telegram holds your taps in between



serverless operation · 2 workflows, Telegram in between

The compose job does the expensive work and ends as soon as the draft is on your phone. The approval job wakes on a short schedule, does nothing if no draft is waiting, and otherwise acts on whatever you tapped. Approve publishes; a revise note starts a fresh compose run (`gh workflow run compose -f feedback=...`).

What has to change in the code:

1. Split `publish_after_approval` into "send draft and save it" and "read the decision and act", exposed as `agent.py --compose` and `agent.py --check-approvals`.
2. Make `wait_for_decision` a single pass over the queued updates instead of a 3 hour loop, with reminders and expiry worked out from the draft's saved send time.
3. At the start of a compose run, read queued `/topic` and direct-post commands; also accept `topic`, `source` and `feedback` as `workflow_dispatch` inputs, so the GitHub mobile app can start a run.
4. Create the progress message in every run, not only in listener mode.

5. Save and load state from the `bot-state` branch (next sections).

Telegram while the job is active

The bot talks to you only while a GitHub job is running; between runs it is silent, and Telegram holds anything you send for up to 24 hours until the next run reads it.

This works because the code already uses Telegram's polling API (`getUpdates`), which needs no public endpoint. Every message and button tap waits in Telegram's queue until some process asks for it.

What you see during a compose run. One progress message, edited in place as the job moves (the `send_progress_tracker` and `update_progress` helpers already exist, but today they only run in listener mode):

1. Run started: daily post, or your topic
2. Researching on the chosen source
3. Drafted N characters, designing the infographic
4. Running 3 reviews and the fact-check (and any retry, with the reason)
5. The infographic and post with Approve and Reject buttons, then the job ends

What happens after you tap. The tap waits in the queue. The next approval run (every 30 minutes in the review window) reads it, changes the button to Posting, publishes, and changes it to Posted with the link. Reject brings up revise, suggest topics or skip; a revise note you type is picked up the same way and starts a fresh compose run with your notes.

Commands when nothing is running. `/topic <text>` or a direct post sent at any time is queued. The next run picks it up, or you can start one at once from the GitHub mobile app (Actions, Run workflow, with a topic field). Set the bot's description to say it replies when a run starts, so silence is expected.

One reader only. `getUpdates` hands each update to whoever asks first. The Mac listener (`setup_mac_bot_daemon.sh`, launchd with KeepAlive) must be unloaded, or it will consume your taps before GitHub sees them.

Latency. GitHub cron runs can start 5 to 30 minutes late at busy times, so an approval may take up to about an hour to post. If you need instant replies, the only fix is a webhook receiver (for example a free Cloudflare Worker that forwards taps to a `repository_dispatch`), which is a tiny hosted function rather than a running server.

State, secrets and cost

Keep all state on a `bot-state` branch in the private repo, pass six secrets, and the whole setup fits in roughly 550 Actions minutes a month, inside the 2,000 free minutes of a private repo on GitHub Free.

State between runs. No job remembers anything, so everything the next run needs is committed to a `bot-state` branch: `posted.json`, `state/drafts.json` (draft id, Telegram message id, status, created time), the pending draft (post text, sources, PNG), and the last Telegram update offset. Today `state/` is gitignored and lost after every CI run. A workflow `concurrency` group stops two jobs writing the branch at once.

Secret	Needed for	Note
`ANTHROPIC_API_KEY`	Research, writing, design, review	Account is out of credits now
`OPENAI_API_KEY`	Fallback model	Missing from the current workflow
`TELEGRAM_BOT_TOKEN`	All bot messages	
`TELEGRAM_CHAT_ID`	Only your taps are accepted	
`LINKEDIN_ACCESS_TOKEN`	Feed post and image upload	Expires about every 60 days; renew with `linkedin_auth.py` on the Mac and update the secret
`LI_AT`	Group posting only	Not recommended in CI (see risks)

The workflow also needs `permissions: contents: write` (state branch) and `actions: write` (an approval run starting a compose run for a revision).

Cost. These are estimates from the current code, not measured runs.

Setup	Minutes per day	Minutes per month
Today's workflow: one job waits up to 3 h for your tap	up to 240	up to 7,200
Proposed: 10 min compose + 8 approval checks of about 1 min	about 18	about 550

GitHub bills every job rounded up to a whole minute, so the approval check must exit in seconds when no draft is pending and run only in the review window, not all day. Installing Playwright adds 1 to 2 minutes, so do it only in a run that actually publishes to groups.

Current state

The pipeline posts successfully, but today it runs from the Mac listener on gpt-4o, and the last post shows the gates are not catching weak content. Evidence is from `bot_listener.log`, `posted.json` and the `output/` folder.

Works

- 3 posts went live on Oct 29 (08:48, 14:59, 15:51), each with an image, and the last one reached all 8 groups.
- The Telegram approval flow works: drafts arrive with buttons, approvals publish once, and buttons change to Posted.
- The source-mode prompt is well designed: one fresh, dated, allow-listed item, with code enforcing the dates and domains rather than trusting the model.
- The SVG route gives sharp, spell-correct infographics, and the code checks block unsafe or unreadable SVG.

Not working

- **Claude is off.** The log shows "Your credit balance is too low" and a switch to gpt-4o. The fallback stays on for the life of the listener process.
- **The 15:51 post was published unsourced.** Its only source was the placeholder `developer.mozilla.org/en-US/docs/Web`, the fact-check said 0 claims could be verified, and it was approved anyway because custom topics ignore the fact-check and get 1 review instead of 3. Its headline was saved as "/topic The architecture shift...".

- **Its infographic has visible defects** that the reviewer scored 5 of 5 for legibility: the badge text runs past its pill, and a checkmark icon sits on top of the words "Error detection".
- **Source comments never post.** LinkedIn returns 403 ACCESS_DENIED for comments, because the Share on LinkedIn product does not include the comments API.
- **The fallback reviewer rejects good dates.** gpt-4o flagged a source as "dated in the future", so source-mode runs fail on the fallback model.
- Of 27 output folders today, 5 reached an approved `post.txt`; most attempts died in review or design.
- There are uncommitted changes (an animated GIF renderer and Telegram edits) and no tests.

Bugs and risks

The two biggest risks are reputational, not technical: unsourced posts going out under your name, and automated group posting that can get the account restricted.

Issue	Where	Effect
Group posting by browser bot, same text to 8 groups 12 s apart, with an automation-hiding flag	`linkedin_groups.py`	Against LinkedIn's rules on automated activity; risk of spam flags or a restricted account. From GitHub's datacenter IPs a login cookie is likely to hit a security check
Custom topics carry a hard-coded fake source and skip the fact-check	`agent.py:389`, `agent.py:543`	Unsourced posts publish with a misleading "Sources" line
Any Anthropic API error counts as a limit (the check matches every `APIStatusError`, and words like "exceeded")	`is_claude_limit_error`, `agent.py:125`	A bad request or auth error silently switches to gpt-4o, and it never switches back
`sys.exit(0)` inside `run()` when no source is left	`agent.py:870`	Kills the whole listener, because the callers only catch `Exception`

Issue	Where	Effect
Emoji top-up appends filler emojis to the ends of paragraphs, and caps at 6 although config allows 7	`clamp_emojis`, `agent.py:575`	The emoji check can never fail; posts get decorative emojis, sometimes on the hashtag line
Your user revisions use up the same 3 attempts as automatic retries	`run()` loop	After a couple of revise rounds the run ends with nothing
`/topic` typed after the Post from Topic button is taken literally	`telegram_review.py:566`	The topic and headline include "/topic"
Telegram messages use `**bold**` with no parse mode	progress and status messages	Asterisks show as literal characters
README is out of date: says Pillow draws the image, `topics.txt` feeds topics (it is never read), `.env.example` exists (it does not)	`README.md`	Setup steps fail for a new reader
Unpinned dependencies; Anthropic search tool pinned to the 2025 version	`requirements.txt`, `config.yaml`	Future installs may break without warning
Secrets live in `.env` beside a committed repo	`.env` is gitignored, good	Keep the repo private: `posted.json` and `config.yaml` are committed

Gaps against the goal, and next steps

Of the three goals, the character limit and the infographic are met; research on a topic you choose is the weak one.

Goal	Status	Gap
Research a private topic	Partly	Topic mode searches only one random allowed source plus 5 fixed sites, and writes from general knowledge when that finds nothing. It cannot read a link you send (no fetch tool) or your own notes or files
Post within the character limit	Met	Code enforces 700 to 3,000. Nothing checks that the hook fits the roughly 200 characters LinkedIn shows before "see more"
Infographic	Met, quality uneven	Rendering is solid; the visual review passes clear overlaps, especially on gpt-4o

Suggested order of work:

- [ ] Top up Anthropic credits, and narrow the fallback to real 429, 529 and credit errors, resetting it at the start of each run
- [ ] Make topic mode research for real: open web search (or a list of trusted domains), a fetch tool for links you send, cited sources required, and the fact-check enforced
- [ ] Turn off group posting, or limit it to groups whose rules allow it and post there by hand
- [ ] Split the pipeline into `--compose` and `--check-approvals` modes and add the two workflows and the `bot-state` branch described above
- [ ] Unload the Mac listener once GitHub takes over
- [ ] Drop the source comment, or show sources in the post body, until the comments API is approved
- [ ] Stop auto-adding emojis; let the check send the draft back instead
- [ ] Pin dependencies, update the README, add a few tests for the code checks and JSON parsing